

Can I memcpy This Type Across a Boundary?

Verifying Object Representation at Compile Time
With C++26 Reflection

FANCHEN SU

Fanchen Su

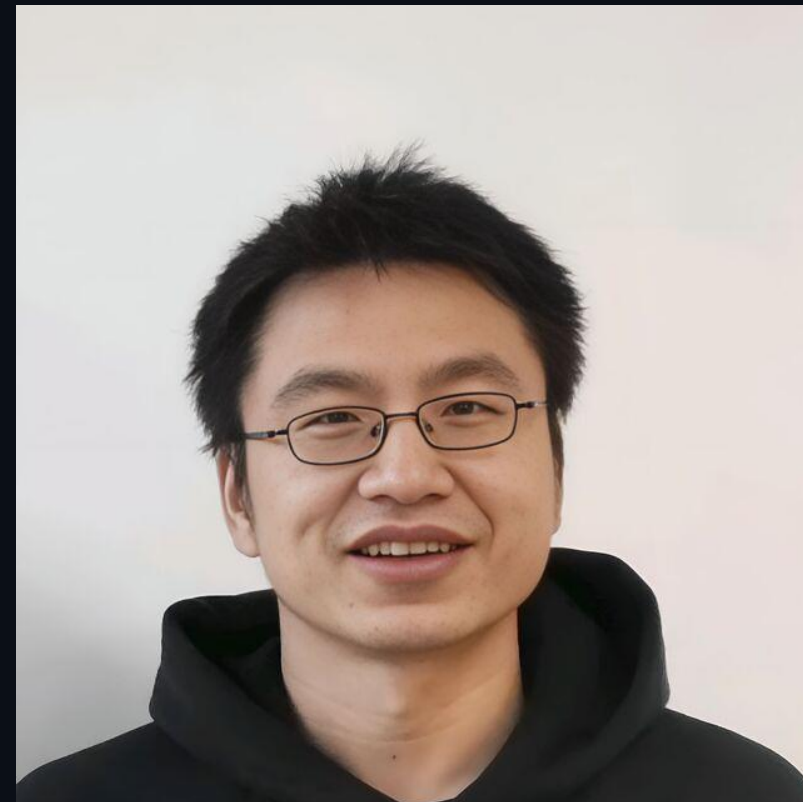
NetEase Games

whucssfc@gmail.com

20+ years of C++ development

Modern C++, performance,
low latency, and maintainability

Ph.D. in Computer Science
Wuhan University, 2012



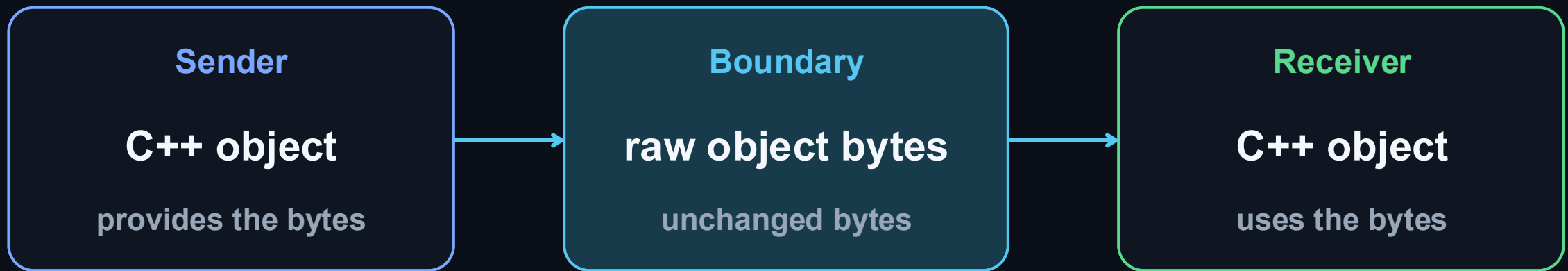
01

What can go wrong across the boundary?

A successful copy is only the start.

We want to pass object bytes across a boundary

Different address space · different build · or both

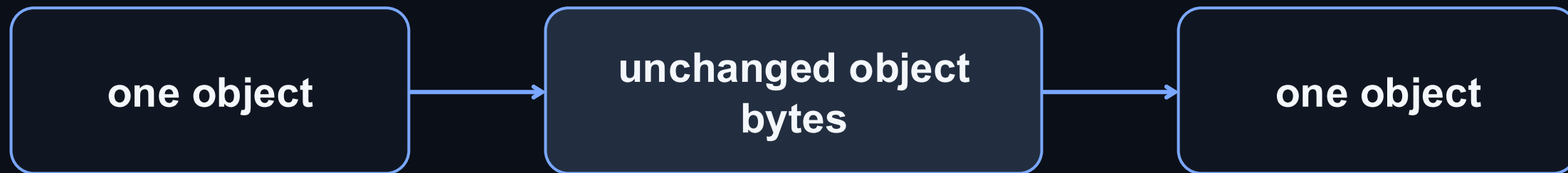


Examples: shared-memory IPC · plugin / host · saved snapshots

Can the receiver use these bytes as a C++ object?

The copied object must work without the original storage

COPY ONE COMPLETE OBJECT



No dependency on objects left in the original storage.

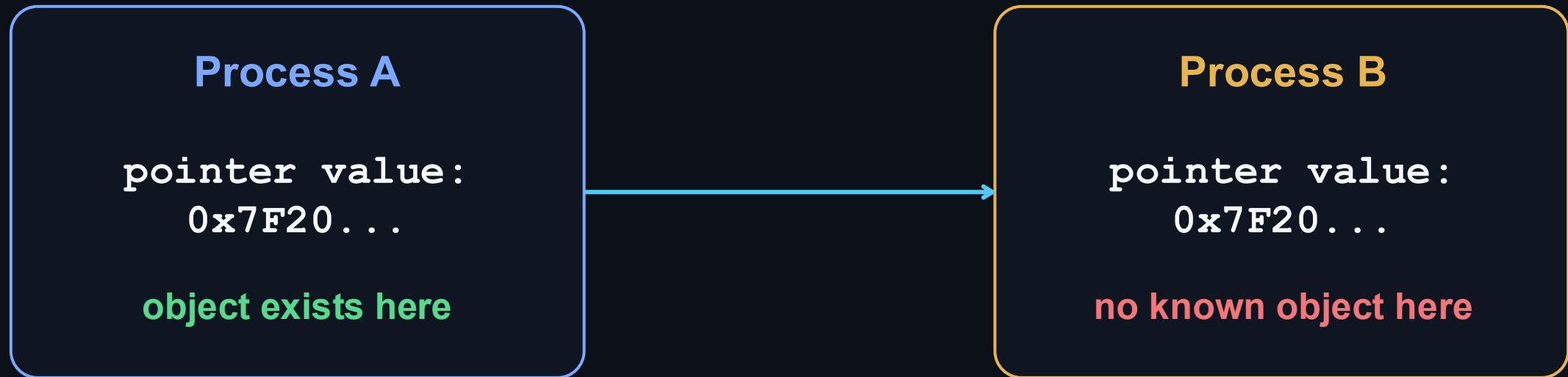
No pointer repair

No field conversion

The app still provides valid storage and object lifetime.

The layout can match while the pointer fails

same build / new process



build: same · address space: different

The number was copied. The pointed-to object was not.

The same field can have different sizes across builds

THIS RECORD · WITHIN EACH BUILD

Valid byte copies reproduce the value.

```
struct Measurement {  
    std::uint64_t id;  
    long double value;  
};
```

FIELD: long double

Linux x86-64

size 16 bytes · alignment 16

ARM64 macOS

size 8 bytes · alignment 8

Same declaration. Different field sizes.

A byte copy does not convert between these layouts.

Two independent checks must pass

1 · BYTE-COPY CHECK

Does the type meet the requirements for this copy?

2 · LAYOUT CHECK

Do the layouts match across these builds?

Matching layouts do not make an old address valid.

Passing the byte-copy check cannot make different layouts match.

The application still decides what the values mean.

A trivially copyable type can still contain pointers

```
struct WithPointer {  
    int* data;  
};  
  
static_assert(  
  
std::is_trivially_copyable_v<WithPointer>);
```

TRAIT: true

BYTE-COPY CHECK: fail

The pointer value is copied too.

For complete objects of this type, valid byte copies reproduce the value.

The trait does not check the target or compare builds.

Manual layout checks need a maintained list

Example: standard-layout T

```
static_assert(
    sizeof(T) == expected_size);
static_assert(
    alignof(T) == expected_alignment);
static_assert(
    offsetof(T, value) == expected_offset);
```

We must maintain the list

nested layout

fields · bases · arrays · bit-fields

number types

integer and floating-point forms

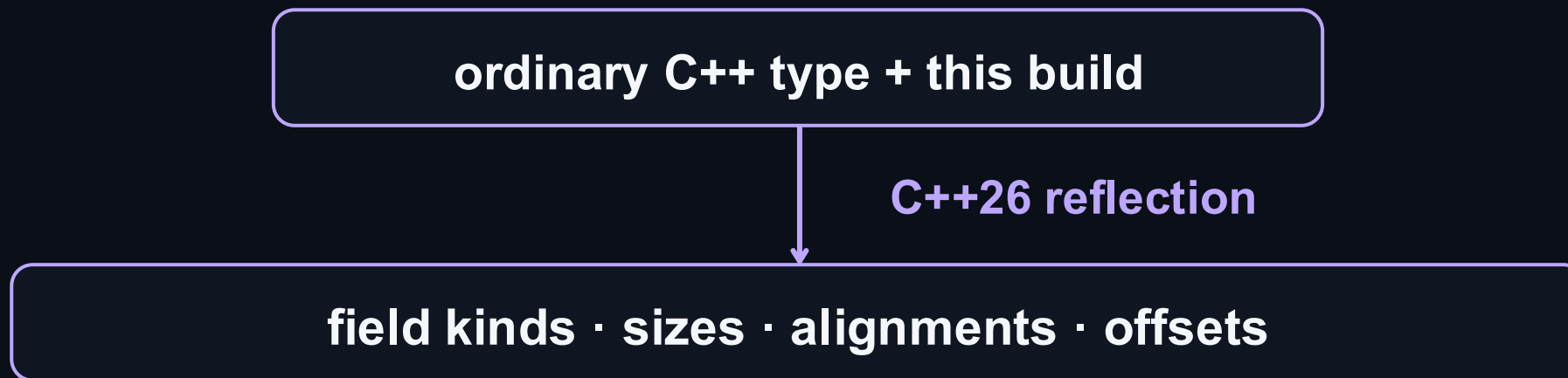
later edits

new or changed members

These checks cover the facts we chose.

The compiler knows the member types and their layout.

Reflection provides evidence for the checks



TYPELAYOUT USES THESE FACTS

BYTE-COPY CHECK

type facts → copy requirements

LAYOUT CHECK

layout facts → build comparison

The answer covers the chosen types, builds, and copy method.

Two failures require two checks

BYTE-COPY CHECK

Does the type meet copy rules?
Independent of old storage

LAYOUT CHECK

Do the recorded layouts match?
Same recorded representation

Both checks · declared types, builds, and copy method

Chapter 2: Let the build report its layout

02

One build reports the layout it made.



A signature tracks the layout facts we compare

01

Record every fact we need.

02

Write the same facts the same way.

03

Change the signature when a recorded fact changes.

04

Stop when a needed fact is hidden.

A signature records the supported representation facts.

Ask the compiler where each field landed

```
struct PacketHeader {  
    std::uint32_t magic;  
    std::uint16_t version;  
    std::uint16_t type;  
    std::uint32_t payload_len;  
    std::uint32_t checksum;  
};
```

THIS BUILD'S BYTE MAP



sizeof = 16

alignof = 4

Do not guess from the field list.
Read what this build produced.

This build reports its map. Every listed build reports its own.

Reflection reads facts. TypeLayout applies rules.

Code excerpt: reading a field's type and byte position

```
using namespace std::meta;  
constexpr auto member = nonstatic_data_members_of(  
    ^^T, access_context::unchecked()) [Index];  
using FieldType = [:type_of(member):];  
constexpr std::size_t field_offset =  
    offset_of(member).bytes + OffsetAdj;
```

list members → **select one field**

read its type → **read its byte position**

OffsetAdj = position of T in the complete object

- **TypeLayout records supported facts.**
- **A special case needs an explicit rule.**
- **A missing fact stops the build.**

Reflection reads the facts. TypeLayout applies the rules.

Field kind matters as well as size

Examples: Linux x86-64 / Clang

C++ type	What the signature keeps	Signature fragment
<code>std::uint32_t</code>	unsigned integer · size 4 · alignment 4	<code>u32[s:4, a:4]</code>
<code>float</code>	floating-point form · size 4 · alignment 4	<code>f32[s:4, a:4]</code>
<code>long double</code>	floating-point form · size 16 · alignment 16	<code>f1d80[s:16, a:16]</code>
object pointer	pointer kind · size 8 · alignment 8	<code>ptr[s:8, a:8]</code>

s = size in bytes · a = alignment in bytes

Same size can still mean a different kind of field.

member names and alias spellings are not stored
Basic-type rules assume these sizes and numeric forms.

Field position within the whole object

source path: outer.inner.value



The signature keeps the final position: byte 12.

Missing layout facts stop the build

```
struct Base { std::uint32_t id; };  
struct Derived : virtual Base {  
    std::uint32_t flags;  
};  
  
constexpr auto signature =  
    boost::typelayout::  
        get_layout_signature<Derived>();
```

TypeLayout rejects virtual inheritance



it cannot finish the signature



stop at compile time

Record what the tool understands.

Name every special case.

Stop when a needed fact is hidden.

This tool does not know enough. Another method may still work.

A layout signature describes one type in one build

LAYOUT FACTS

pointer width	64 bits
byte order	little-endian
object size	16 bytes
object alignment	4 bytes
field positions	0 · 4 · 6 · 8 · 12

EXPORTED LAYOUT SIGNATURE

PacketHeader · Linux x86-64 / Clang

```
[64-le]record[s:16,a:4]{
  @0:u32[s:4,a:4],@4:u16[s:2,a:2],@6:u16[s:2,a:2],
  @8:u32[s:4,a:4],@12:u32[s:4,a:4]}
```

The signature records layout facts, not a CPU name.

One signature describes one type in one build. Compare signatures for the layout check.

Missing facts stop signature generation

```
struct Derived : virtual Base { ... };
```

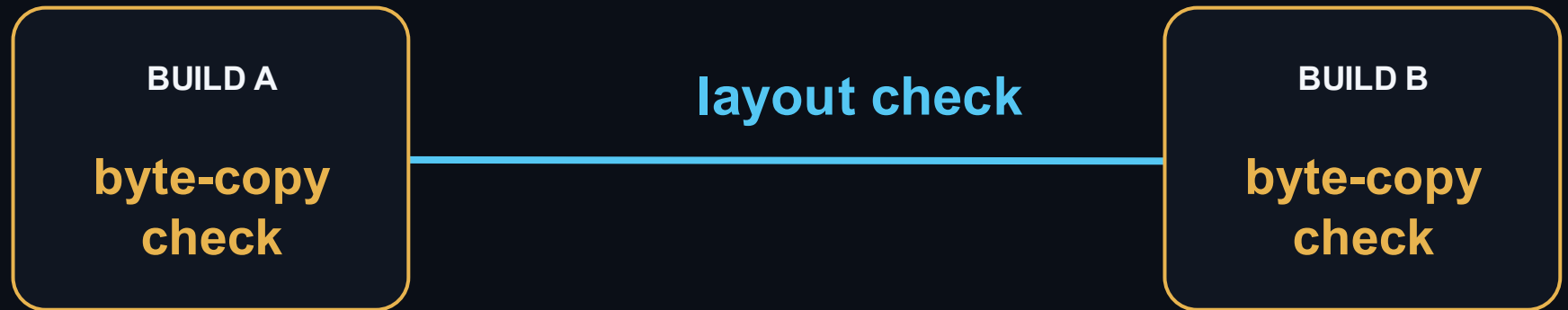
A valid C++ type; unsupported by these signature rules.

Required signature cannot be completed

Stop at compile time. Do not report an incomplete match.

Two checks for one build pair

03



Run the byte-copy check in each build.
Then compare the layouts.

The byte-copy check runs in each build

BYTE-COPY CHECK · ONE OBJECT ON ITS OWN

trivially copyable

no known dependency
on old storage

needed facts and
explicit rules are
available

The result covers one type, one build, and this copy method.

One comparison covers two builds

PacketHeader · complete signatures

compare signatures

Build A · Linux x86-64 / Clang

```
[64-le]record[s:16,a:4]{  
  @0:u32[s:4,a:4],@4:u16[s:2,a:2],  
  @6:u16[s:2,a:2],@8:u32[s:4,a:4],  
  @12:u32[s:4,a:4]}
```

Build B · ARM64 macOS / Clang

```
[64-le]record[s:16,a:4]{  
  @0:u32[s:4,a:4],@4:u16[s:2,a:2],  
  @6:u16[s:2,a:2],@8:u32[s:4,a:4],  
  @12:u32[s:4,a:4]}
```

same type

matching signatures

result: MATCH

Only this type and these two builds.

One field difference rejects the build pair

Measurement · decisive field: long double

Field signature excerpts · size and alignment only

Linux x86-64 size 16 bytes · alignment 16

Linux [s:16,a:16]

ARM64 macOS size 8 bytes · alignment 8

macOS [s:8,a:8]

different field size and alignment

Build comparison:
DIFFER

This pair does not pass.

One different fact is enough.

Both checks in code

IN EACH BUILD · COPY ONE OBJECT ON ITS OWN

```
constexpr auto packet_header_signature =  
    get_layout_signature<PacketHeader>();  
  
static_assert(is_admitted_v<  
    PacketHeader,  
    TransferProfile::ordinary_copy>);
```

`get_layout_signature`

what layout did this build produce?

`is_admitted_v`

does this type pass the byte-copy
check?

LAYOUT CHECK · compare saved layouts in a separate compilation

Linux x86-64 / Clang compared with ARM64 macOS / Clang

```
static_assert(layout_match(linux_plat::PacketHeader_layout,  
                          macos_plat::PacketHeader_layout),  
    "PacketHeader: Linux/macOS layout mismatch!");
```

`layout_match`

do the two saved layouts match?

Enforce both conditions before exchange

LAYOUT CHECK · separate comparison compilation

```
static_assert(layout_match(  
    linux_plat::PacketHeader_layout,  
    macos_plat::PacketHeader_layout),  
    "PacketHeader: Linux/macOS layout mismatch!");
```

Canonical code excerpts · the two checks use separate build contexts

Chapter 4: One pair is not the full answer

04

one pair is only part of the list

listed builds + listed pairs + one way to move the bytes



one final answer

Write down what must work

THE LIST TO CHECK

- 1 — types
- 2 — builds
- 3 — which builds must work together
- 4 — What moves together

One complete object, or one complete buffer

NOT CHECKED

later builds

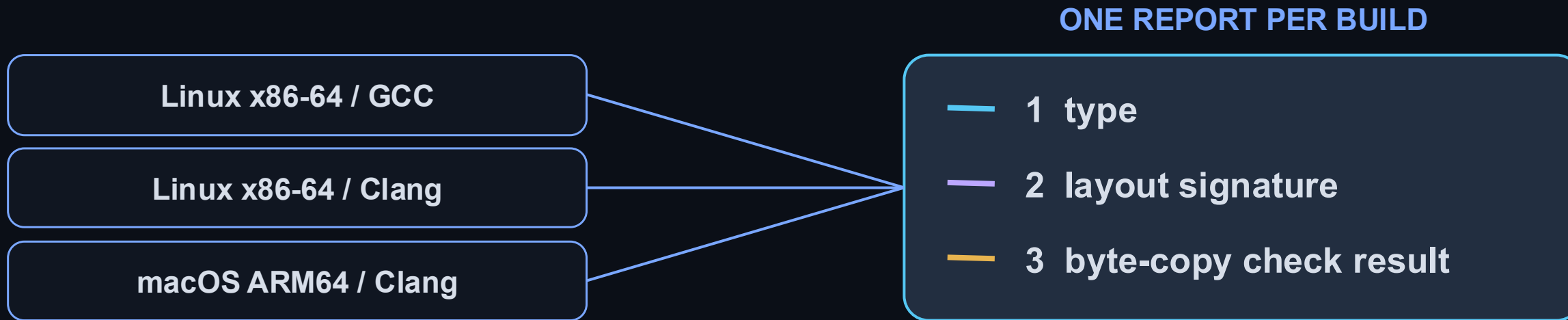
other pairs

other copy
methods

CHECKED listed types · builds · pairs · copy method

CI approves only what is listed.

Each build reports three things



Saved signature · Linux x86-64 / Clang

```
[64-le]record[s:16,a:4]{  
  @0:u32[s:4,a:4],@4:u16[s:2,a:2],  
  @6:u16[s:2,a:2],@8:u32[s:4,a:4],  
  @12:u32[s:4,a:4]}
```

Each report must come from the code and build configuration being checked.

Approve only when the full list passes



One type · one copy method · one build list. Repeat for each type.

Approve only the checked scope

Every required result is present.

Both checks pass for the declared list.

APPROVED WITHIN THIS SCOPE

Representation compatibility has a declared boundary.

Simple cases and a Unit handoff

05

Simple cases show why both checks matter.

A complete Unit applies the same method.

Check the builds, then use the received state.

This fixed-width record passes both checks

COPY ONE OBJECT ON ITS OWN

Linux x86-64 / Clang and ARM64 macOS / Clang

```
struct PacketHeader {  
    std::uint32_t magic;  
    std::uint16_t version;  
    std::uint16_t type;  
    std::uint32_t payload_len;  
    std::uint32_t checksum;  
};
```

BYTE-COPY CHECK

PASS in both builds

Trivially copyable, with no native pointers.

LAYOUT CHECK

MATCH

Recorded layouts match for these builds.

PAIR RESULT: PASS

Both checks pass for this type and build pair.

A native pointer fails the byte-copy check

COPY ONE OBJECT ON ITS OWN

Linux x86-64 / Clang and ARM64 macOS / Clang

```
struct UnsafeWithPointer {  
    std::uint32_t id;  
    char* name;  
    std::uint64_t timestamp;  
};
```

BYTE-COPY CHECK

FAIL: native pointer

The copied address still needs its target.

LAYOUT CHECK

MATCH

Recorded layouts match for these builds.

PAIR RESULT: REJECT

Matching layouts cannot make this pointer independent.

A field difference fails the layout check

COPY ONE OBJECT ON ITS OWN

Linux x86-64 / Clang and ARM64 macOS / Clang

```
struct Measurement {  
    std::uint64_t id;  
    long double value;  
};
```

BYTE-COPY CHECK

PASS in both builds

Trivially copyable, with independent fields.

LAYOUT CHECK

DIFFER: long double

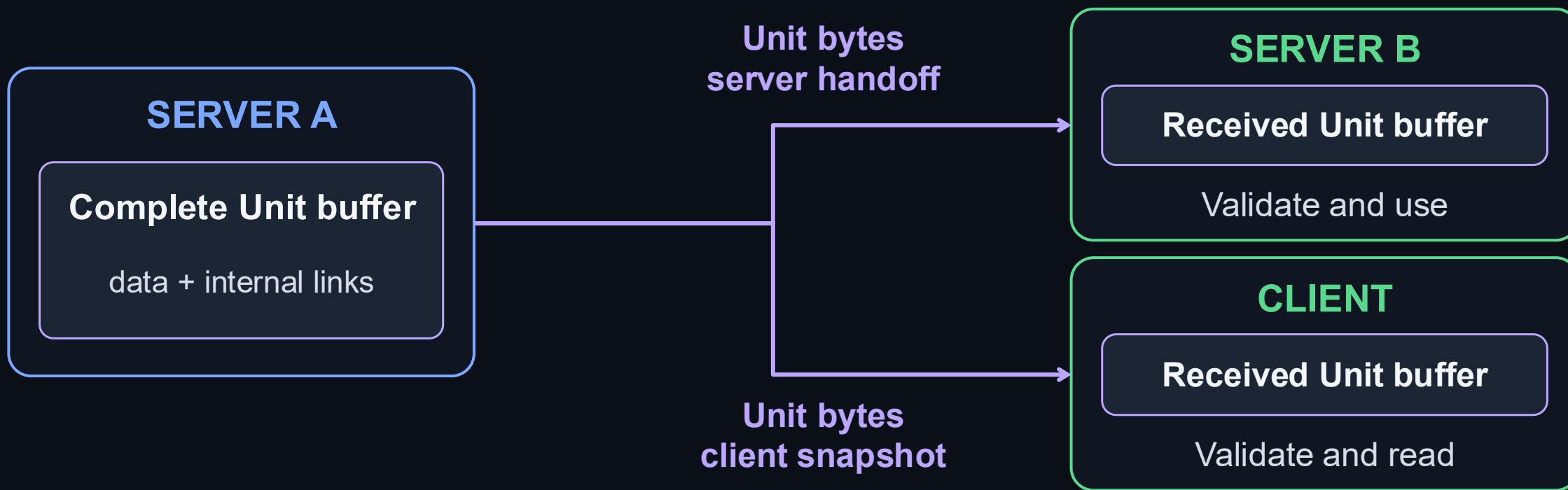
Linux: 16 bytes · macOS: 8 bytes

PAIR RESULT: REJECT

A passing byte-copy check cannot make different layouts match.

A Unit contains a game character's state

ID, position, health, name, effects, and attributes



Also: save the buffer and reload it later.

Receiving types and builds must be on the checked list.

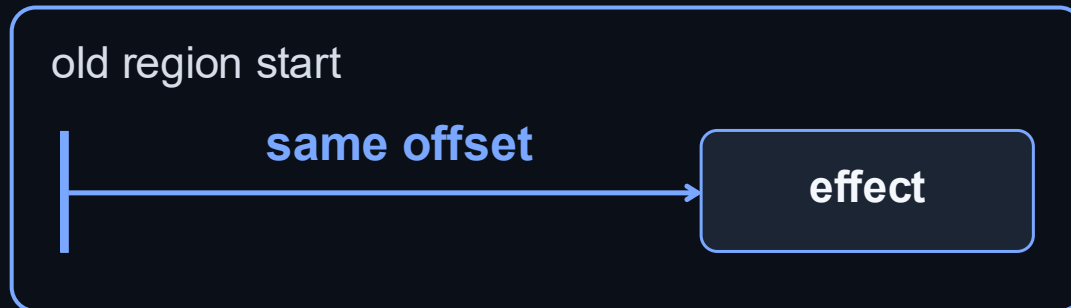
Read the received state without rebuilding each field.

Unit uses internal offsets and external IDs

UNIT · one complete buffer

Inside: relative offsets. Outside: stable IDs for receiver lookup.

BEFORE THE MOVE



AFTER THE MOVE



COPY REQUIREMENTS: full buffer together · no native pointers

new start + same offset = moved target

`relative_ptr` stores an offset from this region's start.

For a valid, non-null link inside the region:

target address = region start + decoded byte offset

Unit passes both checks across six builds

BUILDS CHECKED · 6

Linux x86-64: GCC + Clang Linux ARM64: GCC + Clang
macOS ARM64: Clang macOS x86-64: Clang

UNIT · 4 types

byte-copy check

PASS in every listed build

layout check

MATCH in every required pair

result

UNIT LIST APPROVED

This approval covers the listed Unit types, builds, and copy method.

The receiver still validates the actual bytes.

The receiver attaches Unit and uses its state

Build checks already passed for this type/build/rule list.



AFTER LOAD AND ATTACH

Unit 1001: health 300 → 250

new base · same internal offsets
owner 9001 found · target 2001 absent

Validate the actual bytes before using the objects.

Validate before following links and using state

RECEIVER WORK · after the build checks

1 Check header; copy full buffer

2 Validate offsets and containers

CORRUPT LINK
LOAD
REJECTED

Validation rejects a broken link before the receiver follows it.

What the application must still check

06

Application checks are still required.

The application checks the actual data and its use.

The application checks data, meaning, and use

What remains	Application example or action
Value meaning	Copying a file number does not transfer the file resource.
Valid data and use	Validate the bytes and links. Provide storage, lifetime, and synchronization.
Changed types, builds, or rules	Recheck changes and handle old data. Use a stable format outside the checked list.

A build-time pass does not make every use valid.

Recheck changes and handle old data

WHEN TYPES, BUILDS, OR RULES CHANGE

Run the checks again

Decide how to handle old data

OUTSIDE THE CHECKED LIST

Use a stable data format

Validate its input

A layout match does not establish schema evolution.

Both checks must pass for this boundary

07

FOR THESE TYPES, BUILDS, AND THIS COPY METHOD

IF each type passes the byte-copy check in every build

AND every required pair has matching layouts

THEN the build checks approve this transfer

The application still validates the data and its use.

Reflection supplies facts; the application validates use

Can I memcpy this type across a boundary?

A conditional answer, supported by evidence.

Reflection supplies facts. TypeLayout applies rules.

The application remains responsible for data and use.

Declare the scope. Run both checks. Validate the use.



Thank you

Questions?

github.com/ximicpp/TypeLayout



20
26

